

```
[1] "abc"      "abc.data"  "abtest"    "AMR"
"base64enc"
[6] "bit"      "bit64"     "blob"      "brio"      "bslib"
[11] "cachem"   "callr"     "cli"        "commonmark" "crayon"
[16] "cubature" "DBI"       "desc"       "diffobj"    "digest"
[21] "doParallel" "ellipsis"  "evaluate"   "fansi"
"fastmap"
...
```

Remove

You can remove a package using the `remove.package` function. For example:

```
> remove.packages(c("abc", "abc.data", "abtest"), "/home/
guest/R/x86_64-pc-linux-gnu-library/4.1")
```

The second argument specifies the path where the packages are installed by default. If you install the packages in a different directory, you can set it to the `Library` variable, and use it as the second argument to `remove.packages()`. We can again verify that the packages have been removed:

```
> (.packages(all.available=TRUE))
[1] "AMR"      "base64enc" "bit"      "bit64"
"blob"
[6] "brio"     "bslib"     "cachem"   "callr"
"cli"
[11] "commonmark" "crayon" "cubature" "DBI"
"desc"
[16] "diffobj" "digest" "doParallel" "ellipsis"
"evaluate"
[21] "fansi"    "fastmap" "fontawesome" "foreach"
"fs"
...
```

Update

The `old.packages` function returns a list of packages that have a newer version released in a remote R repository. For example:

```
> old.packages()
      Package  LibPath
Installed
cli      "cli"      "/home/shakthi/R/x86_64-pc-linux-gnu-
library/4.1" "3.1.0"
DBI      "DBI"      "/home/shakthi/R/x86_64-pc-linux-gnu-
library/4.1" "1.1.1"
fansi    "fansi"    "/home/shakthi/R/x86_64-pc-linux-gnu-
library/4.1" "1.0.0"
glue     "glue"     "/home/shakthi/R/x86_64-pc-linux-gnu-
library/4.1" "1.4.2"
```

```
...
      Built  ReposVer  Repository
cli      "4.1.2" "3.1.1"    "https://cloud.r-project.org/src/
contrib"
DBI      "4.1.0" "1.1.2"    "https://cloud.r-project.org/src/
contrib"
fansi    "4.1.2" "1.0.2"    "https://cloud.r-project.org/src/
contrib"
glue     "4.1.0" "1.6.1"    "https://cloud.r-project.org/src/
contrib"
...
```

The `cli.3.1.0` package has been installed on the system, but there is a newer '3.1.1' package available. We can update the package using the `update.packages` function, as follows:

```
> update.packages()
cli :
  Version 3.1.0 installed in /home/shakthi/R/x86_64-pc-linux-
gnu-library/4.1
  Version 3.1.1 available at https://cloud.r-project.org
Update? (Yes/no/cancel) Yes
DBI :
  Version 1.1.1 installed in /home/shakthi/R/x86_64-pc-linux-
gnu-library/4.1
  Version 1.1.2 available at https://cloud.r-project.org
Update? (Yes/no/cancel) no
...
** building package indices
** testing if installed package can be loaded from temporary
location
** checking absolute paths in shared objects and dynamic
libraries
** testing if installed package can be loaded from final
location
** testing if installed package keeps a record of temporary
installation path
* DONE (cli)
```

The command prompts you for a confirmation before proceeding to update the package. You can choose not to update a specific package, or cancel the entire operation. In the above example, we only update the `cli` package to 3.1.1. The same can be verified using the `packageVersion` function, as follows:

```
> packageVersion("cli")
[1] '3.1.1'
```

Download

The `new.packages` function shows the uninstalled packages from the local system that are available in remote R